

PROJECT DOCUMENTATION

NEOCHAT: REAL-TIME FULL-STACK MESSAGING PLATFORM

A Store-and-Forward Architectural Solution for Data Sovereignty and Zero Message Loss

Submitted by: Mukesh Balaji

Internship Track: Full-Stack Web Development & Engineering

Organization: Blackbucks Engineering & Technology Internship

Technology Stack: React.js, Spring Boot, PostgreSQL, WebSockets

Date: July 2026

Document Version: 1.0 (Final Comprehensive Edition)

DECLARATION OF AUTHORSHIP

I, Mukesh Balaji, hereby declare that the project titled 'NeoChat: Real-Time Full-Stack Messaging Platform' and the documentation presented in this report constitute an original piece of software engineering completed under the guidance of Blackbucks Internship Mentors.

This platform has been designed, implemented, and thoroughly tested in accordance with modern software architecture standards. All data models, REST endpoints, real-time messaging protocols, and automated unit/integration test suites detailed herein reflect actual source code and empirically verified execution logs.

I confirm that this document complies strictly with all project guidelines, formatting constraints, and technical specifications mandated by Blackbucks, including Times New Roman typography, 14pt bold headings, 12pt body text, and complete exclusion of hyperlinks and underlined text.

ACKNOWLEDGEMENT

I extend my deepest gratitude to the Blackbucks Internship Team for providing a structured and rigorous environment to build real-world software engineering solutions. The hands-on project rounds — ranging from system design and API integration to automated testing and documentation — have enriched my comprehension of full-stack engineering.

I would also like to thank the open-source communities behind Spring Boot, React.js, PostgreSQL, SockJS, and Vitest, whose robust frameworks enabled the creation of this enterprise-grade communication application.

TABLE OF CONTENTS

Declaration of Authorship

Acknowledgement

Executive Summary

Chapter 1: Project Overview & Scope

Chapter 2: Problem Statement & Technical Objectives

Chapter 3: System Requirements Specification

Chapter 4: High-Level System Architecture & Protocol Flow

Chapter 5: Backend Service Micro-Architecture & Data Modeling

Chapter 6: Frontend Architecture & User Interface Design

Chapter 7: Real-Time Messaging Engine & Delivery State Machine

Chapter 8: Automated Testing, Quality Assurance & Verification

Chapter 9: Deployment, Containerization & Self-Hosting Guidelines

Chapter 10: Conclusion & Future Architectural Enhancements

Appendix A: Complete REST API & DTO Specifications

Appendix B: Comprehensive Test Case Matrix

EXECUTIVE SUMMARY

Instant messaging has become the foundational backbone of corporate communication, social interaction, and operational collaboration. However, contemporary commercial chat applications such as WhatsApp, Slack, and Telegram rely on closed, centralized infrastructure that introduces critical vulnerabilities: silent message loss during network drops, lack of delivery transparency, and absence of data sovereignty.

The NeoChat platform addresses these fundamental limitations by introducing a self-hostable, full-stack messaging platform engineered on a store-and-forward architectural model. Developed using React.js for the frontend and Java Spring Boot for the backend, supported by a PostgreSQL relational database, NeoChat guarantees zero message loss across all network conditions.

Every message transmitted across the system is immediately persisted to disk before network delivery is attempted. Connected clients communicate via bidirectional, authenticated WebSockets (STOMP over SockJS), receiving instant delivery when online. When a recipient is disconnected, messages are queued deterministically and delivered in exact chronological order upon reconnection.

Furthermore, NeoChat introduces a multi-state delivery system (Pending, Delivered, Read) broadcasted to the sender in real-time. This documentation details the end-to-end design, implementation, and verification of the system, supported by an automated test suite of 82 passing test cases across both frontend and backend modules.

CHAPTER 1: PROJECT OVERVIEW & SCOPE

1.1 Background & Context

Over the past decade, the reliance on digital real-time messaging has grown exponentially. Organizations require secure, resilient, and audit-friendly communication tools to manage remote workflows and sensitive intellectual property. Traditional proprietary messaging platforms store data on third-party servers under black-box security models, creating compliance and privacy risks.

NeoChat was conceived as an open, self-hostable alternative that empowers individuals and enterprise organizations to maintain complete control over their messaging infrastructure while providing a sleek, modern user experience.

1.2 Project Scope

The scope of the NeoChat project encompasses the complete software development lifecycle, including requirements gathering, architecture modeling, full-stack engineering, API design, database schema creation, real-time protocol integration, automated unit and integration testing, and documentation.

- **Frontend Development:** Building an intuitive, responsive Neumorphic web application in React.js supporting user authentication, real-time messaging, contact lists, and profile management.
- **Backend Engineering:** Constructing a robust Spring Boot microservice architecture providing RESTful APIs, Spring Security with JSON Web Tokens (JWT), and WebSocket broker configuration.
- **Persistence & Reliability:** Designing a relational schema in PostgreSQL using Hibernate ORM to store users, conversation histories, and delivery states.
- **Quality Assurance:** Developing automated unit, controller, and integration tests using JUnit 5, Mockito, MockMvc, and Vitest.

1.3 Core Deliverables

The key deliverables produced during this project include:

- Fully functional Spring Boot backend application providing authentication, user management, and WebSocket messaging.
- Modern React single-page application integrated with backend REST endpoints and STOMP WebSocket channels.
- Comprehensive test suite consisting of 39 backend tests and 43 frontend tests with 100% pass rates.

- Complete system documentation, setup instructions, and deployment guides.

The following sections establish the technical foundation and problem analysis that drove the design of NeoChat.

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Limitations of Existing Messaging Platforms

Centralized commercial communication platforms have achieved widespread global adoption due to high availability and low latency. However, these systems exhibit three fundamental engineering shortcomings:

2.1.1 Silent Message Loss During Offline Periods

In conventional peer-to-peer or loosely persisted chat architectures, messages sent to offline recipients are frequently held in ephemeral memory buffers. If the underlying server restarts or network connection drops before the recipient reconnects, the message is permanently lost without notifying either party. Senders assume the message was delivered, leading to operational breakdowns.

2.1.2 Lack of Delivery Transparency

Many existing messaging platforms provide vague or unreliable status indicators. Senders are unable to determine whether a delay is caused by server processing failure, client disconnection, or recipient inactivity. This lack of granular state visibility creates communication ambiguity.

2.1.3 Absence of Self-Hosting and Data Sovereignty

Proprietary chat applications force organizations to transmit sensitive communications through central servers owned by external service providers. Regulated industries (such as healthcare, finance, and defense) cannot utilize such systems due to strict data compliance and sovereignty laws.

2.2 Technical Objectives

To overcome these technical obstacles, NeoChat was engineered around seven core technical objectives:

- **Guarantee Zero Message Loss:** Implement a Store-and-Forward model that writes every message to disk before initiating network transfer.
- **Provide Sub-Second Real-Time Communication:** Establish persistent bidirectional WebSockets for instant message delivery between connected clients.
- **Deliver Transparent Multi-State Tracking:** Implement a 3-state lifecycle (Pending, Delivered, Read) with real-time status updates broadcasted to senders.
- **Ensure Self-Hostability:** Package the application so that any organization can deploy it on internal infrastructure with zero external cloud dependencies.

- Implement Enterprise Security: Protect user credentials with BCrypt hashing and secure stateless JWT authentication.
- Deliver a Premium User Interface: Build an aesthetically pleasing Neumorphic UI in React.js supporting dark/light visual modes.
- Maintain Rigorous Test Coverage: Validate system stability using automated unit, integration, and UI tests across all software layers.

CHAPTER 3: SYSTEM REQUIREMENTS SPECIFICATION

This section formalizes the functional and non-functional requirements governing the implementation of NeoChat.

3.1 Functional Requirements

The functional requirements specify the exact capabilities and behaviors expected from the software system.

3.1.1 User Authentication & Session Management

- FR-01 User Registration: Users must be able to register with a unique username, email address, and password.
- FR-02 Password Security: Passwords must be salted and hashed using BCrypt prior to storage.
- FR-03 User Login: Registered users must authenticate using valid credentials to receive a signed JWT session token.
- FR-04 Persistent Sessions: Client application must maintain authentication state across browser reloads using secure local storage.
- FR-05 WebSocket Authentication: WebSocket connection handshakes must validate JWT session tokens before granting channel subscriptions.

3.1.2 Real-Time Messaging & Storage

- FR-06 Send Message: Authenticated users must be able to send direct text messages to any registered user.
- FR-07 Database Persistence: Every message must be written to PostgreSQL with timestamp and Pending status prior to network dispatch.
- FR-08 Real-Time Broadcast: Active WebSocket connections must receive instant message payloads when recipient is online.
- FR-09 Offline Message Queuing: Messages for offline recipients must remain stored in DB and fetched in exact chronological sequence upon login.
- FR-10 Conversation History: Users must be able to retrieve historical chat conversations via authenticated REST endpoints.

3.1.3 Multi-State Delivery Tracking

- FR-11 Pending State: Message receives Pending status immediately upon database insertion.

- FR-12 Delivered State: Message updates to Delivered status when recipient client confirms receipt.
- FR-13 Read State: Message updates to Read status when recipient opens the active chat window.
- FR-14 Real-Time Status Propagation: Status transitions must be broadcast to the sender in real-time via WebSockets.

3.1.4 User & Contact Management

- FR-15 Contact Listing: Users must be able to view all registered users in a contact directory.
- FR-16 Live Presence Tracking: System must broadcast online/offline presence changes to all connected users.
- FR-17 Profile Editing: Users must be able to update their username, email address, and password.

3.2 Non-Functional Requirements

- NFR-01 Reliability & Durability: Zero message loss under network disruption due to store-and-forward architecture.
- NFR-02 High Performance: Message latency between online clients must remain below 100 milliseconds.
- NFR-03 Security & Privacy: All API routes must enforce Bearer token validation and reject unauthenticated requests.
- NFR-04 Scalability: Stateless JWT architecture allows horizontal backend scaling across multiple node instances.
- NFR-05 Usability: Responsive, accessible UI supporting dynamic search, unread badges, and visual feedback.

CHAPTER 4: HIGH-LEVEL SYSTEM ARCHITECTURE

NeoChat utilizes a decoupled full-stack architecture separating the client-side single page application (React.js) from the server-side microservices (Spring Boot) and persistence store (PostgreSQL).

4.1 System Component Overview

The architecture is structured across three core layers:

- **Presentation Layer (Client):** React single-page application managing UI rendering, local state, authentication context, REST API requests, and WebSocket event subscriptions.
- **Application Service Layer (Server):** Java Spring Boot container hosting REST Controllers, Security Filters, JWT Provider, Business Logic Services, and WebSocket STOMP Message Broker.
- **Persistence Layer (Database):** PostgreSQL relational database maintaining User accounts, Hashed Passwords, Conversation Histories, and Message Delivery States.

4.2 Communication Protocols

NeoChat employs two complementary communication channels:

- **HTTP REST Protocol:** Used for stateless operations such as user registration, login, profile updates, contact retrieval, and conversation history fetching.
- **STOMP over SockJS WebSocket Protocol:** Used for full-duplex, low-latency communication including real-time message delivery, status updates, and user presence broadcasts.

4.3 Decoupled Network Flow

When a user sends a message, the following sequence occurs:

1. Client sends a JSON payload to the REST endpoint `/api/messages/send` or WebSocket destination `/app/chat.sendMessage`.
2. Spring Security JWT Filter interceptor validates the Bearer token in the request header.
3. `MessageService` saves the message to PostgreSQL with status `PENDING`.
4. If the recipient is connected, `SimpMessagingTemplate` dispatches the payload to `/topic/messages/{recipientId}`.
5. Recipient client acknowledges receipt, triggering a status update to `DELIVERED`.
6. When recipient views the message, status updates to `READ`, and a status frame is sent back to the sender.



BLACKBUCKS

ENGINEERING & TECHNOLOGY INTERNSHIP

CHAPTER 5: BACKEND SERVICE MICRO-ARCHITECTURE & DATA MODELING

The backend of NeoChat is engineered using Java 21 and Spring Boot 3.4.1, leveraging Spring Security, Spring Data JPA, and Hibernate ORM.

5.1 Relational Database Schema Design

The database consists of two primary relational entities stored in PostgreSQL: users and messages.

5.1.1 Users Entity (`users`)

- `id` (UUID, Primary Key): Unique identifier generated automatically.
- `user_name` (VARCHAR, Unique, Not Null): User's handle.
- `email` (VARCHAR, Unique, Not Null): User's email address.
- `hashed_password` (VARCHAR, Not Null): BCrypt encrypted password string.
- `is_online` (BOOLEAN, Not Null): Live presence status flag.
- `created_at` (TIMESTAMP, Not Null): Automatic creation timestamp.
- `updated_at` (TIMESTAMP, Nullable): Automatic update timestamp.

5.1.2 Messages Entity (`messages`)

- `id` (UUID, Primary Key): Unique message identifier.
- `content` (TEXT, Not Null): Text message payload.
- `sender_id` (UUID, Foreign Key → `users.id`): Reference to sender.
- `recipient_id` (UUID, Foreign Key → `users.id`): Reference to recipient.
- `time_stamp` (TIMESTAMP, Not Null): Creation timestamp.
- `status` (VARCHAR, Not Null): Enum value (PENDING, DELIVERED, READ).

5.2 Service Layer Implementation

The backend logic is encapsulated across three service classes:

- `AuthService`: Manages user registration, BCrypt password matching, token generation via `JwtService`, and user session logout.
- `MessageService`: Handles store-and-forward message persistence, conversation history retrieval using custom JPQL queries, and delivery status transitions.

- UserService: Manages user directory listings, profile updates, and secure password changes.

CHAPTER 6: FRONTEND ARCHITECTURE & USER INTERFACE DESIGN

The frontend of NeoChat is built as a single-page application using React.js and Vite, featuring a modern Neumorphic visual design system.

6.1 Application Structure & Routing

The client application uses react-router-dom to manage four core page views:

- LoginPage (`/login`): Form for user authentication with error handling and link to registration.
- RegisterPage (`/register`): Form for new account creation with username, email, and password fields.
- ChatPage (`/chat`): Main messaging hub displaying user list, active chat window, and message input bar.
- ProfilePage (`/profile`): User account overlay for updating profile details, changing passwords, and signing out.

6.2 State Management & Context API

Global authentication state is managed via AuthContext.jsx. The context stores currentUser, token, and provides login(), register(), logout(), and updateUser() functions.

Tokens and user credentials are saved in localStorage (neochat_token and neochat_user) to preserve session state across page refreshes.

6.3 Neumorphic UI Design System

The user interface implements a Neumorphic design language defined in neu.css. Soft inset and drop shadows create tactile, extruded UI elements, providing a sleek aesthetic for buttons, input fields, cards, and message bubbles.

CHAPTER 7: REAL-TIME MESSAGING ENGINE & DELIVERY STATE MACHINE

A distinguishing feature of NeoChat is its transparent multi-state message delivery tracking system.

7.1 Message Delivery State Machine

Every message transitions through three explicit states:

- State 1: PENDING — Assigned immediately when the server receives the message and persists it to the database.
- State 2: DELIVERED — Assigned when the recipient's client receives the message payload via WebSocket or REST history sync.
- State 3: READ — Assigned when the recipient opens the conversation and views the message on screen.

7.2 Real-Time Status Propagation

Status changes are updated in the database by `MessageService.updateStatus()` and emitted instantly to the sender's client via WebSocket. `MessageBubble.jsx` renders distinct visual icons for each status: single checkmark for Pending, double checkmarks for Delivered, and double blue checkmarks for Read.

7.3 Offline Message Synchronization

When a user reconnects after being offline, `apiService.getHistory(contactId)` retrieves all missed messages from PostgreSQL. Messages with status PENDING automatically transition to DELIVERED, providing seamless sync.

CHAPTER 8: AUTOMATED TESTING, QUALITY ASSURANCE & VERIFICATION

To ensure system stability, security compliance, and regression resistance, NeoChat includes a comprehensive automated test suite consisting of 82 test cases.

8.1 Backend Automated Testing (39 Tests Passed)

Backend tests were developed using JUnit 5, Mockito, and Spring MockMvc across five test classes:

- `AuthControllerTest` (6 Tests): Validates REST controllers, payload validation (400 Bad Request), and registration/login HTTP responses (200 OK).
- `AuthServiceTest` (8 Tests): Tests registration, BCrypt password encoding, login verification, duplicate checks, and logout state changes.
- `MessageServiceTest` (8 Tests): Tests message creation, store-and-forward persistence, status transitions, and conversation history fetching.
- `UserServiceTest` (11 Tests): Tests contact filtering, profile modifications, password change security, and online status mapping.
- `JwtServiceTest` (5 Tests): Tests HMAC-SHA256 token generation, UUID extraction round-trips, and signature tampering detection.
- `BackendApplicationTests` (1 Test): Verifies full Spring Boot context loading.

8.2 Frontend Automated Testing (43 Tests Passed)

Frontend tests were executed using Vitest and React Testing Library across seven test files:

- `apiService.test.js` (16 Tests): Mocks HTTP fetch requests to test REST integration, Bearer header injection, token storage, and response mapping.
- `LoginPage.test.jsx` (5 Tests): Validates UI elements, input bindings, and navigation links.
- `RegisterPage.test.jsx` (4 Tests): Validates registration form inputs and submit actions.
- `ChatPage.test.jsx` (3 Tests): Validates auth routing protection and main chat interface.
- `ProfilePage.test.jsx` (5 Tests): Validates profile edit actions, password updates, and sign-out controls.
- `UserList.test.jsx` (5 Tests): Validates search filtering, contact selection, and unread badges.
- `MessageBubble.test.jsx` (5 Tests): Validates sent/received bubble rendering, timestamp formatting, and status icons.

Overall, all 82 tests achieved a 100% pass rate with zero failures.



BLACKBUCKS

ENGINEERING & TECHNOLOGY INTERNSHIP

CHAPTER 9: DEPLOYMENT, CONTAINERIZATION & SELF-HOSTING GUIDELINES

NeoChat is designed for straightforward self-hosting on private servers, virtual private servers (VPS), or local infrastructure.

9.1 Prerequisites

- Java Development Kit (JDK) 21 or later.
- Node.js 18+ and npm package manager.
- PostgreSQL relational database server.

9.2 Backend Deployment Steps

1. Configure database connection parameters in `backend/src/main/resources/application.yml`.
2. Build executable JAR file: `./mvnw clean package -DskipTests`.
3. Execute production application: `java -jar target/backend-0.0.1-SNAPSHOT.jar`.

9.3 Frontend Deployment Steps

1. Install dependencies: `npm install`.
2. Configure reverse proxy in `vite.config.js` to route ``/api`` and ``/ws`` to ``localhost:8080``.
3. Launch development server (`npm run dev`) or build static bundle (`npm run build`).

CHAPTER 10: CONCLUSION & FUTURE ARCHITECTURAL ENHANCEMENTS

10.1 Project Summary

The NeoChat project successfully delivers a full-stack real-time messaging solution that resolves the core issues of silent message loss, delivery opacity, and platform lock-in. By coupling a React.js presentation layer with a Java Spring Boot backend and PostgreSQL database, NeoChat guarantees reliable store-and-forward message delivery and multi-state tracking.

10.2 Future Enhancement Roadmap

Future development iterations will expand upon the current architecture through the following technical additions:

- End-to-End Encryption (E2EE): Implementing Signal Protocol for client-side encryption.
- Group Messaging & Rooms: Extending database schema and STOMP channels to support multi-user chat rooms.
- Media & Attachment Support: Integrating AWS S3 / MinIO object storage for image and file sharing.
- Voice & Video Calling: Integrating WebRTC protocols for peer-to-peer audio and video calls.

APPENDIX A: COMPLETE REST API SPECIFICATIONS

The following table documents all production REST endpoints exposed by the NeoChat Spring Boot backend:

Method	Endpoint URL	Auth Required	Description
POST	/api/auth/register	No	Registers new user account with BCrypt password hashing
POST	/api/auth/login	No	Authenticates credentials and returns JWT Bearer token
POST	/api/auth/logout	Yes	Invalidates session and sets user offline
GET	/api/users	Yes	Fetches directory of registered users excluding self
GET	/api/users/{id}	Yes	Retrieves user details by UUID
PUT	/api/users/profile	Yes	Updates current user username and email
PUT	/api/users/password	Yes	Changes user password after old password validation
POST	/api/messages/send	Yes	Persists message to DB with Pending status and dispatches
GET	/api/messages/{userId}	Yes	Retrieves conversation history between two users
PUT	/api/messages/{id}/status	Yes	Updates message status (Pending -> Delivered -> Read)

APPENDIX B: AUTOMATED TEST EXECUTION SUMMARY

Summary of automated test suites executed across backend and frontend codebases:

Suite Name		Framework	Total Tests	Pass Rate
Backend Controller	Auth	Spring MockMvc	6	100% Pass (6/6)
Backend Service	Auth	JUnit 5 + Mockito	8	100% Pass (8/8)
Backend Service	Message	JUnit 5 + Mockito	8	100% Pass (8/8)
Backend Service	User	JUnit 5 + Mockito	11	100% Pass (11/11)
Backend Service	JWT	JUnit 5	5	100% Pass (5/5)
Backend Test	Context	SpringBootTest	1	100% Pass (1/1)
Frontend API Service		Vitest	16	100% Pass (16/16)
Frontend UI Pages		Vitest + RTL	17	100% Pass (17/17)
Frontend Components		Vitest + RTL	10	100% Pass (10/10)